



Homework 8
—
**Artificial Neural Networks
(ANN)**



Introduction

Artificial neural networks (ANN) are computing systems vaguely inspired by the biological neural networks that constitute animal brains. Such systems "learn" to perform tasks by considering examples, generally without being programmed with task-specific rules. ANNs are primarily used to classification and regression problems. For example, in image recognition, they might learn to identify images that contain cats by analyzing example images that have been manually labeled as "cat" or "no cat" and using the results to identify cats in other images (supervised learning / classification). They do this without any prior knowledge of cats, for example, that they have fur, tails, whiskers and cat-like faces. Instead, they automatically generate identifying characteristics from the examples that they process.

An ANN is based on a collection of connected nodes called artificial neuron. Each connection, like the synapses in a biological brain, can transmit a signal to other neurons. An artificial neuron that receives a signal then processes it and can signal neurons connected to it.

In ANN implementations, the "signal" at a connection is a real number, and the output of each neuron is computed by some non-linear function of the sum of its inputs. The connections are called edges. Neurons and edges have a weight that adjusts as learning proceeds. The weight increases or decreases the strength of the signal at a connection. Neurons may have a threshold such that a signal is sent only if the aggregate signal crosses that threshold. Typically, neurons are aggregated into layers. Different layers may perform different transformations on their inputs. Signals travel from the first layer (the input layer), to the last layer (the output layer), possibly after traversing the layers multiple times.

The original goal of the ANN approach was to solve problems in the same way that a human brain would. But over time, attention moved to performing specific tasks, leading to deviations from biology. ANNs have been used on a variety of tasks, including computer vision, speech recognition, machine translation, social network filtering, playing board and video games, medical diagnosis, and even in activities that have traditionally been considered as reserved to humans, like painting.

[source: https://en.wikipedia.org/wiki/Artificial_neural_network]



History of ANN

Warren McCulloch and Walter Pitts (1943) opened the subject by creating a computational model for neural networks. Rosenblatt (1958) created the perceptron. The first functional networks with many layers were published by Ivakhnenko and Lapa in 1965. The basics of continuous backpropagation (the primary means of training an ANN) were derived in the context of control theory by Kelley in 1960 and by Bryson in 1961.

In 1973, Dreyfus used backpropagation to adapt parameters of controllers in proportion to error gradients. Werbos's (1975) backpropagation algorithm enabled practical training of multi-layer networks. In 1982, he applied Linnainmaa's AD method to neural networks in the way that became widely used. Thereafter research stagnated following Minsky and Papert (1969), who discovered that basic perceptrons were incapable of processing the exclusive-or circuit and that computers lacked sufficient power to process useful neural networks. In 1992, max-pooling was introduced to help with least-shift invariance and tolerance to deformation to aid 3D object recognition. Schmidhuber adopted a multi-level hierarchy of networks (1992) pre-trained one level at a time by unsupervised learning and fine-tuned by backpropagation.

The development of metal–oxide–semiconductor (MOS) very-large-scale integration (VLSI), in the form of complementary MOS (CMOS) technology, enabled the development of practical artificial neural networks in the 1980s. In 2012, Ng and Dean created a network that learned to recognize higher-level concepts, such as cats, only from watching unlabeled images. Unsupervised pre-training and increased computing power from GPUs and distributed computing allowed the use of larger networks, particularly in image and visual recognition problems, which became known as "deep learning".

Ciresan and colleagues (2010) showed that despite the vanishing gradient problem, GPUs make backpropagation feasible for many-layered feedforward neural networks. Between 2009 and 2012, ANNs began winning prizes in ANN contests, approaching human level performance on various tasks, initially in pattern recognition and machine learning. For example, the bi-directional and multi-dimensional long short-term memory (LSTM) won three competitions in connected handwriting recognition in 2009 without any prior knowledge about the three languages to be learned.

Ciresan and colleagues built the first pattern recognizers to achieve human-competitive/superhuman performance on benchmarks such as traffic sign recognition (IJCNN 2012).
[source: https://en.wikipedia.org/wiki/Artificial_neural_network]



The Artificial Neuron – Basic Structure

An artificial neuron is a mathematical function conceived as a model of biological neurons, a neural network. Artificial neurons are elementary units in an artificial neural network. The artificial neuron receives one or more inputs and sums them to produce an output (or activation). Usually each input is separately weighted, and the sum is passed through a non-linear function known as an activation function (often called transfer function). The activation functions usually have a sigmoid shape, but they may also take the form of other non-linear functions, piecewise linear functions, or step functions. They are also often monotonically increasing, continuous, differentiable and bounded. [source: https://en.wikipedia.org/wiki/Artificial_neuron]

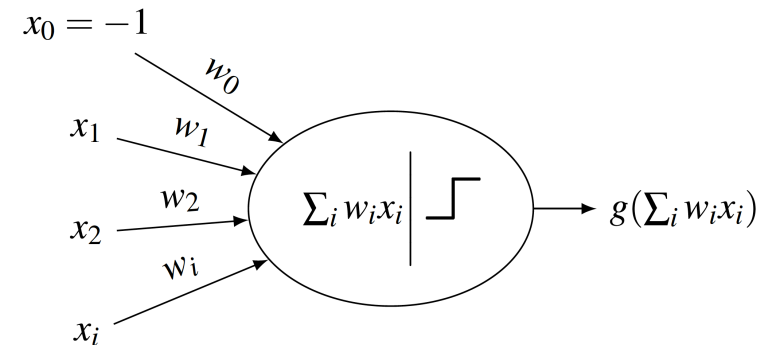
For a given artificial neuron, let there be $m + 1$ inputs with signals x_0 through x_m and weights w_0 through w_m . Usually, the x_0 input is assigned the value -1 (or $+1$), which makes it a bias input with $-w_0$ (or $+w_0$). This leaves only m actual inputs to the neuron: from x_1 to x_m .

The output of the neuron is:
$$y = g\left(\sum_{i=0}^m w_i x_i\right)$$

Where g is the activation function (commonly a threshold function).

The output is analogous to the axon of a biological neuron, and its value propagates to the input of the next layer, through a synapse. It may also exit the system, possibly as part of an output vector.

It has no learning process as such. Its transfer function weights are calculated and threshold value are predetermined.

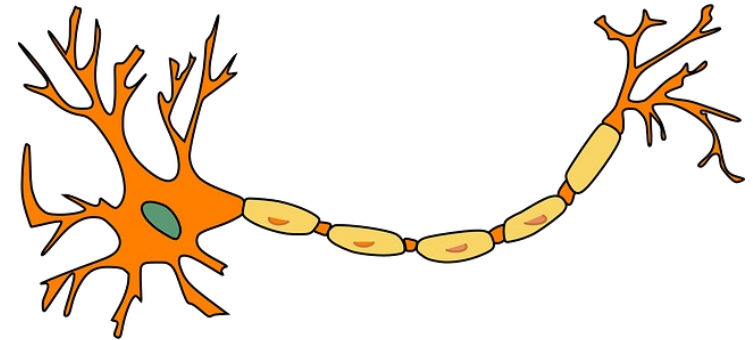




The Artificial Neuron – Biological models

Artificial neurons are designed to mimic aspects of their biological counterparts.

- ♦ **Dendrites:** In a biological neuron, the dendrites act as the input vector. These dendrites allow the cell to receive signals from a large (>1000) number of neighboring neurons. As in the above mathematical treatment, each dendrite can perform "multiplication" by that dendrite's "weight value." The multiplication is accomplished by increasing or decreasing the ratio of synaptic neurotransmitters to signal chemicals introduced into the dendrite in response to the synaptic neurotransmitter. A negative multiplication effect can be achieved by transmitting signal inhibitors (i.e., oppositely charged ions) along the dendrite in response to the reception of synaptic neurotransmitters.
- ♦ **Soma:** In a biological neuron, the soma acts as the summation function, seen in the above mathematical description. As positive and negative signals (exciting and inhibiting, respectively) arrive in the soma from the dendrites, the positive and negative ions are effectively added in summation, by simple virtue of being mixed in the solution inside the cell's body.
- ♦ **Axon:** The axon gets its signal from the summation behavior which occurs inside the soma. The opening to the axon essentially samples the electrical potential of the solution inside the soma. Once the soma reaches a certain potential, the axon will transmit an all-in signal pulse down its length. In this regard, the axon behaves as the ability for us to connect our artificial neuron to other artificial neurons.



Unlike most artificial neurons, however, biological neurons fire in discrete pulses. Each time the electrical potential inside the soma reaches a certain threshold, a pulse is transmitted down the axon. This pulsing can be translated into continuous values. The rate (activations per second, etc.) at which an axon fires converts directly into the rate at which neighboring cells get signal ions introduced into them. The faster a biological neuron fires, the faster nearby neurons accumulate electrical potential (or lose electrical potential, depending on the "weighting" of the dendrite that connects to the neuron that fired). It is this conversion that allows computer scientists and mathematicians to simulate biological neural networks using artificial neurons which can output distinct values (often from -1 to 1).



Artificial Neurons for Boolean Function

The AND Function

With a single artificial neuron, we can implement some simple functions already. Let us use this model of an artificial neuron to implement the Boolean AND function. Let the activation function g be a threshold function that returns 0 if the weighted sum of the input values is less than zero and 1 if it is greater than zero.

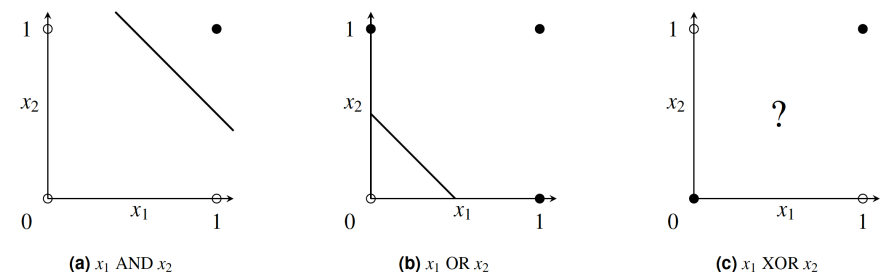
For the Boolean AND function, two input variables x_1 and x_2 are required, each of which can have the values 0 or 1. The result is 1 if both inputs are 1, otherwise 0. Now the weights must be set so that the desired result is achieved: $w_0 = 1.5, w_1 = 1, w_2 = 1$.

The weight w_0 acts here as a threshold value (bias). This is due to the activation function used at zero; Since w_0 is subtracted from the weighted sum by the fixed input $x_0 = -1$, a weight not equal to zero shifts the threshold value to any other position, here to 1.5. Any other threshold value between 1 and 2 can also be selected for the AND function.

Other Functions

Other functions can be similarly implemented if they are linearly separable. The weights describe nothing more than a hyperplane (in 2D, as in the example, a straight line) with which the separation between the desired output values zero or one is achieved. Thus, the Boolean OR can easily be implemented, but not the Boolean XOR as shown in the figure.

Learning or training now means that the weights are automatically determined from an annotated sample.





Artificial Neurons for Boolean Function

Estimated time: 10min for the questions

♦ **Please answer the following questions**

- 1) What does an artificial neuron look like to implement the Boolean OR function?
- 2) What does an artificial neuron look like to implement the Boolean NOT function?



Structure of ANN

Estimated time: 20min to watch the videos + 30min for the questions

Now that we understand the artificial neuron as a basic building block, we will see how to combine multiple neurons into an artificial neural network. There is a great animated introduction to neural networks by Grant Sanderson, who is running the YouTube channel **3Blue1Brown**.

I can highly recommend all his videos; they are fun to watch but still go deep into the underlying (mathematical) concepts. This first video will introduce the basic concepts of artificial neural networks. You already know a bit more about "what a neuron is" than he assumes, but this will only help you better understand the video.

- ♦ **Watch the Video “But what is a Neural Network?”**

<https://www.youtube.com/watch?v=aircAruvnKk>

- ♦ **Please answer the following questions**

- 1) How many input neurons and how many output neurons does the example used in the video have? Why exactly this many?
- 2) What range of values can the activation of each neuron take on?
- 3) Does it matter, in which order the pixels of the input image are fed into the network? Why/Why not?
- 4) He chose 2 hidden layers with 16 neurons each, admitting that this is a quite arbitrary choice. What do you expect to happen, if the number of hidden layers and the number of neurons in them is increased?
- 5) Why does your network with 784 input neurons and 16 neurons in the second layer have 12560 weights between these layers, and not $12544 = 784 \cdot 16$?
- 6) What is the motivation given for having exactly 2 hidden layers?
- 7) In the video when moving to the matrix/vector multiplication, he is keeping the biases as a separate vector (added on). In the first Homework Notebook, we saw the trick to treat the bias just like all other weight. How do we need to change the matrix W and the activation vector a to eliminate the bias vector b from the formula?
- 8) In summary, what is a neural network?



Gradient Descent (revisited)

Estimated time: 17min to watch the videos + 30min for the questions

After watching the first video, you have a basic idea about how ANNs work. What you still do not know is how to set all these thousands or millions of parameters programmatically. If you think back to the very first class, you may remember that most machine learning methods consist of three parts: a model, a training algorithm and a model evaluation algorithm. In the case of ANN,

- **the model** is the structure chose (number of hidden layers, number of neurons, activation function, etc.) and the set of weights (incl. the biases).
- **the model evaluation algorithm** is the computation shown in the previous video: starting at the input layer, computing all the matrix multiplications and activation (squishification) functions and outputting the values of the output layer. This is the so called "feed forward" algorithm.
- **the training algorithm** for a neural network computes all the weights and biases - naturally, this is hard, as there are so many of them. The most widely used algorithm is called "backpropagation" (backprop for short), and you will learn about the underlying idea and the algorithm the following video. It is a gradient descent based algorithm, you already know about gradient descent from a previous class.

- ♦ **Watch the Video “Gradient descent, how neural networks learn” up to time index 16:40** (naturally, you can watch the rest of the video as well if you are curious)

<https://www.youtube.com/watch?v=IHZwWFHWa-w>

- ♦ **Please answer the following questions**

- 1) How do we initialize the weights and biases of a network at the start of the training phase? How good do we expect this network to perform?
- 2) Why do we need a "cost function"?
- 3) How is the "cost of one training example" defined mathematically?
- 4) What does it mean of this cost is small, what if is large?
- 5) How do we compute the cost of ALL the training examples? What is the meaning of this value?
- 6) What does the Gradient of a multidimensional function tell you? How does this help us in minimizing the cost function?
- 7) What do we mean mathematically, if we talk about "a neural network is learning"?
- 8) What is "Gradient Descent"?
- 9) Why does a neural network give confident answers for the classification problem? How could you try to solve this problem?



Congratulations!

You are now familiar with the basic structure of ANN and the basic idea of the backpropagation algorithm.

In the lecture, we will be looking at these in more detail.